

Task 1 SRAM PUF Design: Technical Explanation of `puf_module.v`

Study and defense note

June 2, 2026

Contents

1	Scope and High-Level Architecture	2
2	Interface Signals and Module Responsibilities	2
2.1	UART-facing signals	2
2.2	RAM-facing signals	2
2.3	Connection to <code>combined_ram</code>	3
3	Byte Indexing Model	3
4	State Encoding and Persistent Registers	4
5	Initialization and Reset Behavior	5
6	Sequential Logic	5
6.1	INIT	5
6.2	WAIT_FOR_REQUEST	6
6.3	WAITCYCLE_FOR_MEMORY	6
6.4	PUF_READ	6
6.5	UART_SEND	6
6.6	UART_WAIT_FINISH	6
6.7	LOOP_CONDITION	7
7	Combinational Logic	7
7.1	Safe defaults	7
7.2	INIT combinational behavior	8
7.3	WAIT_FOR_REQUEST combinational behavior	8
7.4	WAITCYCLE_FOR_MEMORY and PUF_READ	8
7.5	UART_SEND	8
7.6	UART_WAIT_FINISH and LOOP_CONDITION	8
8	PUF-UART Handshake in Detail	8
9	Diagnostic Mode and Observed Debugging Interpretation	9
9.1	Final debugging observations	9
10	Build and BRAM Initialization Note	10
11	Measurement and Analysis Notes	10
12	Short Defense Summary	11

1 Scope and High-Level Architecture

This document explains the logic of the Task 1 SRAM PUF design from the point of view of the `puf_module.v` controller. The goal is not to compute a PUF value in software, but to read the power-up contents of SRAM/BRAM, transmit them to a PC over UART, and later evaluate the collected data statistically.

At the architectural level, the design connects three components:

PC script $\longleftrightarrow$ UART $\longleftrightarrow$ puf_module FSM $\longleftrightarrow$ combined_ram / BRAM

The Python script sends a one-byte request over UART. The FPGA receives this request, starts an FSM-controlled memory readout, and returns a byte stream. In normal mode, the request byte is `s` or `S`; in temporary debug mode, `d` or `D` can be used to return diagnostic bytes.

The normal design target is:

- read the 16 KiB memory space exposed through `combined_ram`;
- treat the RAM output as 16-bit words;
- split each 16-bit word into two 8-bit UART bytes;
- send 16384 bytes to the PC;
- save this as 512 lines, where each line contains 64 hexadecimal characters, corresponding to one 256-bit PUF instance.

Thus, `puf_module` can be understood as a small data mover: it bridges a word-addressed memory interface and a byte-oriented UART interface.

2 Interface Signals and Module Responsibilities

2.1 UART-facing signals

The module has the following UART-facing interface:

```
input  uart_rx_ready,
input  [7:0] uart_data_from_rx,
input  uart_tx_ready,
output [7:0] uart_data_to_tx,
output uart_tx_enable
```

`uart_rx_ready` indicates that the UART RX module has received a valid byte from the PC. That byte is available on `uart_data_from_rx`. In this design, the FSM waits for `s/S` to start a normal PUF readout, or `d/D` to start a temporary diagnostic packet.

`uart_tx_ready` indicates that the UART TX module can accept a new byte. The FSM only starts a transmit operation when this signal is high.

`uart_data_to_tx` is the byte presented to the UART transmitter. `uart_tx_enable` is a one-cycle pulse telling the transmitter to accept and send that byte. In the final implementation, this enable pulse is generated as a registered sequential pulse rather than as a purely combinational state output. This prevents the transmitter from sampling the same byte more than once.

2.2 RAM-facing signals

The RAM interface consists of:

```
wire [15:0] rdata;
reg  [12:0] raddr;
reg  [12:0] waddr;
reg  [15:0] wmask;
```

```
reg          we;
reg [15:0] wdata;
```

`rdata` is a 16-bit data word returned by `combined_ram`. It is declared as a `wire` because it is driven by the RAM wrapper and observed by `puf_module`. The read address `raddr` is 13 bits because the memory space contains 8192 16-bit words:

$$8192 \times 16 \text{ bits} = 131072 \text{ bits} = 16384 \text{ bytes} = 16 \text{ KiB.}$$

The write-side signals `waddr`, `wdata`, `wmask`, and `we` are normally inactive in the final PUF readout, because writing would destroy the power-up memory contents. They were only used in a temporary writeback experiment to test whether the RAM write/read path was alive. For the final PUF capture, this writeback path is disabled.

2.3 Connection to `combined_ram`

The instance is conceptually simple:

```
combined_ram ram_inst (
    .clk      ( clk ),
    .rdata    ( rdata ),
    .raddr    ( raddr ),
    .we       ( we ),
    .wdata    ( wdata ),
    .wmask    ( wmask ),
    .waddr    ( waddr )
);
```

From the perspective of `puf_module`, `combined_ram` behaves like a single 8192-word memory. Internally, however, `combined_ram` maps the address onto 32 physical `SB_RAM40_4K` blocks. The upper bits select the block, and the lower bits select the local word address:

Address field	Meaning	Example role
<code>raddr[12:8]</code>	RAM block index	selects one of 32 blocks
<code>raddr[7:0]</code>	local word offset	selects one word within the block

The controller does not manually select individual physical RAMs. It simply drives `raddr` and observes `rdata`. The final design was also verified with the original `combined_ram.v`; the functional fixes were made in `puf_module.v`, not in the RAM wrapper.

3 Byte Indexing Model

A key design choice is to use `i_r` as a byte index, while the RAM is word-addressed. Since the RAM returns 16-bit words and the UART sends 8-bit bytes, each RAM word must be split into two transmitted bytes.

The relationship is:

Byte index <code>i_r</code>	RAM word address <code>raddr</code>	Selected byte
0	0	<code>rdata[7:0]</code>
1	0	<code>rdata[15:8]</code>
2	1	<code>rdata[7:0]</code>
3	1	<code>rdata[15:8]</code>
$\vdots$	$\vdots$	$\vdots$
16382	8191	<code>rdata[7:0]</code>
16383	8191	<code>rdata[15:8]</code>

This is implemented by:

```
raddr = i_r[13:1];
```

and by selecting the low or high byte using `i_r[0]`:

```
if (i_r[0] == 1'b0)
    puf_byte_reg <= rdata[7:0];
else
    puf_byte_reg <= rdata[15:8];
```

A concise defense statement is:

I use `i_r` as the UART byte index. Since the RAM is 16-bit word-addressed, `i_r[13:1]` becomes the RAM word address, and `i_r[0]` selects either the low or high byte from `rdata`.

4 State Encoding and Persistent Registers

The FSM states are defined with `localparam`:

```
localparam
    INIT = 0,
    WAIT_FOR_REQUEST = 1,
    WAITCYCLE_FOR_MEMORY = 2,
    PUF_READ = 3,
    UART_SEND = 4,
    UART_WAIT_FINISH = 5,
    LOOP_CONDITION = 6;

reg [2:0] state;
```

`localparam` gives symbolic names to numeric state values. There are seven states, and therefore a three-bit register is sufficient, because three bits can represent eight values.

The main persistent registers are:

Register	Purpose
<code>state</code>	current FSM state
<code>i_r</code>	read byte index, normally 0 to 16383
<code>i_w_r</code>	temporary debug write address counter
<code>puf_byte_reg</code>	stable one-byte UART transmit buffer
<code>debug_mode</code>	selects normal PUF stream or diagnostic stream
<code>uart_seen_busy</code>	records that UART TX has entered busy state after a send pulse

`puf_byte_reg` is especially important: it decouples the RAM output from the UART input. The FSM first captures a selected RAM byte into this register, and only in the next state does it pulse the UART transmit enable. This keeps the UART data stable during the transmit handshake.

The final implementation also uses `uart_seen_busy` to make the UART transmit handshake more robust. During debugging, a repeated-byte problem was observed in which diagnostic marker bytes appeared as 44 44 42 42 instead of 44 42. This showed that the transmitter could sample the enable condition more than once. The final design therefore treats `uart_tx_enable` as a registered one-clock pulse and waits until `uart_tx_ready` first goes low and then returns high before sending the next byte.

5 Initialization and Reset Behavior

The reset branch initializes the FSM and its internal control registers:

```
state <= INIT;
i_r <= 14'd0;
i_w_r <= 13'd0;
debug_mode <= 1'b0;
uart_seen_busy <= 1'b0;
uart_tx_enable <= 1'b0;
puf_byte_reg <= 8'd0;
```

The reset does *not* initialize the SRAM/BRAM contents. That distinction is essential: the task is based on reading the memory power-up state. Resetting the controller is allowed; overwriting the PUF memory is not.

The intended meaning of each initialization is:

- `state <= INIT`: start from the known first FSM state.
- `i_r <= 0`: begin every capture from byte index zero.
- `i_w_r <= 0`: keep the temporary debug write counter deterministic.
- `debug_mode <= 0`: return to normal capture mode after reset.
- `uart_seen_busy <= 0`: clear the UART busy-observation flag.
- `uart_tx_enable <= 0`: ensure that no transmit pulse is active after reset.
- `puf_byte_reg <= 0`: ensure the UART transmit buffer starts from a known value.

A useful explanation is:

Reset only initializes the controller. It does not intentionally initialize the PUF memory, because the memory power-up state is the object being measured.

6 Sequential Logic

The sequential block is sensitive to the positive clock edge and reset:

```
always @ (posedge clk, posedge rst) begin
    ...
end
```

This block contains the FSM state transitions and all persistent register updates. Non-blocking assignments are used because all register updates should conceptually occur together on a clock edge.

At the beginning of every non-reset cycle, the final implementation clears the registered transmit enable:

```
uart_tx_enable <= 1'b0;
```

Only states that intentionally start a UART transmission override this default and assert `uart_tx_enable` for one clock cycle.

6.1 INIT

INIT is a short setup state. In the sequential block, it may generate one registered dummy transmit pulse and then transitions to `WAIT_FOR_REQUEST`. The dummy byte is not PUF data; it is only a startup/cleanup byte for the serial connection.

6.2 WAIT_FOR_REQUEST

In `WAIT_FOR_REQUEST`, the FSM remains idle until the UART RX side reports a received command byte. The read index is reset to zero so that every new capture starts from the beginning of memory.

The request handling is:

- `s/S`: start normal PUF dump;
- `d/D`: start diagnostic packet mode.

Temporary debug writeback logic may also advance `i_w_r` while idle. This is not required by the final task; it is only a diagnostic method for testing the write/read path. In the final PUF measurement configuration, this writeback is disabled by setting `DEBUG_TEST_WRITEBACK = 1'b0`.

6.3 WAITCYCLE_FOR_MEMORY

This state is a timing spacer. The address `raddr` is driven from `i_r` by combinational logic. Since the RAM primitive is clocked, the FSM waits before sampling `rdata`. The state therefore exists mainly to respect memory latency.

6.4 PUF_READ

In this state, the FSM captures one byte into `puf_byte_reg`. In normal mode, the byte comes from `rdata`. In diagnostic mode, the byte comes from a debug case statement that returns known markers and selected internal signal values.

After loading `puf_byte_reg`, the FSM checks `uart_tx_ready`. If the transmitter is ready, the FSM clears `uart_seen_busy` and proceeds to `UART_SEND`.

The timing is important:

At the clock edge in `PUF_READ`, `puf_byte_reg` receives the selected byte. In the following state, `UART_SEND`, this stable byte is presented to the UART transmitter.

6.5 UART_SEND

`UART_SEND` is a one-cycle transmit-pulse state. In the final implementation, `uart_tx_enable` is not generated as a purely combinational output. Instead, it is registered in the sequential FSM:

```
UART_SEND :
begin
    uart_tx_enable <= 1'b1;
    state <= UART_WAIT_FINISH;
end
```

At the beginning of every non-reset clock cycle, the FSM assigns:

```
uart_tx_enable <= 1'b0;
```

Thus, `uart_tx_enable` becomes a true one-clock pulse. This change was important in the final debugging process, because a combinational state-based enable caused diagnostic bytes to appear twice in the UART stream.

6.6 UART_WAIT_FINISH

After a transmit pulse, the UART needs many FPGA clock cycles to serialize the byte. A naive implementation would leave this state as soon as `uart_tx_ready` is high again. However, during debugging it was observed that `uart_tx_ready` may still be high immediately after the send pulse. If the FSM accepts that high value too early, it can move on before the UART has clearly entered its busy phase.

Therefore, the final implementation uses a two-phase wait:

1. first wait until `uart_tx_ready` becomes low, proving that the UART accepted the byte and entered busy state;
2. then wait until `uart_tx_ready` becomes high again, proving that the byte transmission has completed.

This is implemented with `uart_seen_busy`:

```
if (!uart_tx_ready) begin
    uart_seen_busy <= 1'b1;
end else if (uart_seen_busy) begin
    uart_seen_busy <= 1'b0;
    state <= LOOP_CONDITION;
end
```

This prevents the FSM from launching the next byte too early.

6.7 LOOP_CONDITION

This state decides whether the transfer is complete. In normal mode, the transfer ends after `PUF_BYTES` bytes. The optional diagnostic mode can also stream data for debugging, but the RAM writeback path is disabled during final PUF measurements.

If the transfer is not complete, `i_r` is incremented and the FSM returns to `WAITCYCLE_FOR_MEMORY`. Thus, the main loop is:

```
WAITCYCLE_FOR_MEMORY -> PUF_READ -> UART_SEND -> UART_WAIT_FINISH -> LOOP_CONDITION
```

7 Combinational Logic

The combinational block generates current-cycle control and output signals based on the current FSM state and register values:

```
always @ (*) begin
    ...
end
```

The main principle is to assign safe defaults first and override them only in states that need special behavior.

7.1 Safe defaults

The default assignments are conceptually:

```
uart_data_to_tx = puf_byte_reg;
raddr = i_r[13:1];
wmask = 0;
waddr = 13'd0;
wdata = 16'd0;
we = 1'b0;
```

These defaults mean:

- keep the UART data connected to the stable byte buffer;
- continuously derive the RAM read address from the byte index;
- do not write to RAM unless explicitly enabled for debugging.

In the final implementation, `uart_tx_enable` is deliberately excluded from the combinational defaults. It is generated as a registered one-clock pulse in the sequential FSM. This avoids repeated sampling of the same transmit byte by the UART module.

7.2 INIT combinational behavior

In INIT, the combinational block may still drive the dummy data byte:

```
uart_data_to_tx = 8'h00;
```

However, the transmit enable itself is generated in the sequential INIT state as a registered pulse. This keeps the transmit-enable behavior consistent with the normal UART_SEND state.

7.3 WAIT_FOR_REQUEST combinational behavior

In normal mode, WAIT_FOR_REQUEST keeps the safe defaults. In temporary debug writeback mode, it can drive:

```
waddr = i_w_r;
wdata = 16'hA5A5;
we = 1'b1;
```

This is a diagnostic experiment. It attempts to overwrite RAM with a visible pattern before readout. If readback works, the output should contain A5 bytes. If readback remains zero, the issue is likely in the RAM path or primitive behavior rather than in UART formatting. In the final PUF measurement configuration, this path must remain disabled.

7.4 WAITCYCLE_FOR_MEMORY and PUF_READ

These states typically keep the defaults in the combinational block. `raddr` is already driven from `i_r`, and the actual sampling of `rdata` happens in the sequential PUF_READ state.

7.5 UART_SEND

In the final implementation, UART_SEND does not assert `uart_tx_enable` combinationaly. The combinational block only keeps `uart_data_to_tx` connected to `puf_byte_reg`. The actual one-clock transmit pulse is generated in the sequential FSM.

7.6 UART_WAIT_FINISH and LOOP_CONDITION

These states keep the defaults. UART_WAIT_FINISH should not assert `uart_tx_enable`; it waits until the UART transmitter becomes busy and then ready again. LOOP_CONDITION only updates counters and state in the sequential block.

8 PUF-UART Handshake in Detail

The complete normal transfer starts when the PC sends `s`. The UART RX module captures this byte, and `puf_module` observes `uart_rx_ready` and `uart_data_from_rx`. Once the command is recognized, the FSM begins the memory read loop.

For each byte, the sequence is:

1. `i_r` determines the RAM word address through `raddr = i_r[13:1]`.
2. The FSM waits one memory cycle.
3. In PUF_READ, it selects either `rdata[7:0]` or `rdata[15:8]` and stores it in `puf_byte_reg`.
4. In UART_SEND, `uart_data_to_tx` is stable and a registered `uart_tx_enable` pulse is asserted for one clock cycle.
5. In UART_WAIT_FINISH, the FSM first waits for `uart_tx_ready` to go low, and then waits for it to return high.

6. In `LOOP_CONDITION`, the FSM either increments `i_r` and repeats, or returns to idle after the final byte.

The transmit side is therefore a ready/enable handshake:

Condition	Action	Purpose
<code>uart_tx_ready = 1</code> before send <code>UART_SEND</code>	enter <code>UART_SEND</code> registered one-clock <code>uart_tx_enable</code> pulse	transmitter can accept a byte start transmission
<code>UART_WAIT_FINISH</code>	wait for ready low, then ready high	ensure byte was accepted and completed

This handshake is necessary because the FPGA clock is much faster than the UART baud rate. A byte cannot be sent on every clock cycle. The FSM must wait for the UART transmitter to finish serializing one byte before launching the next one.

9 Diagnostic Mode and Observed Debugging Interpretation

The temporary d/D request mode was introduced to distinguish between three possible problems:

- UART or Python capture failure;
- FSM sequencing failure;
- RAM or BRAM readout failure.

The diagnostic packet contains known marker bytes such as `44 42` (ASCII DB) and `A5 5A C3 3C`. If these appear in the PC capture, then the FSM entered debug mode and the UART path is capable of carrying non-zero bytes.

If diagnostic marker bytes are visible but the `rdata` fields remain `00 00`, then the evidence points away from UART/Python and toward the RAM output path, BRAM contents, or the `combined_ram` interface behavior.

An important alignment issue was also considered: in a full 16 KiB diagnostic capture, marker bytes can be searched explicitly using a deliberate marker such as `44 42`. For normal PUF streams, however, skipping initial zeros would be unsafe, because a real PUF response may legitimately start with zero bytes.

9.1 Final debugging observations

The most important debugging observation was that the diagnostic marker initially appeared as:

`44 44 42 42`

instead of the intended:

`44 42`

Since these bytes are constants generated by the debug case statement, this duplication could not be explained by RAM contents or address mapping. It indicated a UART transmit handshake problem: the same byte was effectively being transmitted twice.

The fix was to change `uart_tx_enable` from a combinational state output into a registered one-clock pulse generated in the sequential FSM. After this change, the diagnostic stream started correctly with `44 42`, and the byte duplication disappeared.

A temporary RAM writeback test was also used. With `DEBUG_TEST_WRITEBACK = 1`, the design wrote `16'hA5A5` into RAM while idle. Reading back `A5A5` proved that the `combined_ram` read/write path and UART output path were functional. For the final PUF capture, this debug writeback was disabled:

```
localparam DEBUG_TEST_WRITEBACK = 1'b0;
```

After disabling the writeback and power-cycling the board, the design produced non-constant, random-looking PUF data. The same behavior was also confirmed with the original `combined_ram.v`, meaning that the final functional fix was in `puf_module.v`, not in the provided RAM wrapper.

10 Build and BRAM Initialization Note

The assignment requires packing the bitstream in a way that avoids initializing BRAM to zero. The relevant `icepack` option is:

```
icepack -n top_level.asc top_level_noinit.bin
```

The `-n` option means “skip initializing BRAM”. This is conceptually separate from `READ_MODE` and `WRITE_MODE` parameters in the RAM primitive. Those mode parameters configure RAM behavior; they do not by themselves mean that memory contents are initialized to zero.

A typical manual build flow is:

```
yosys -q \
  -L top_level_yosys.log \
  -p "hierarchy -top top_level" \
  -p "synth_ice40 -json top_level.json" \
  rtl/top_level.v rtl/uart.v rtl/puf_module.v rtl/combined_ram.v

nextpnr-ice40 -q \
  --hx8k --package ct256 \
  --freq 12 \
  -l top_level_nextpnr.log \
  --json top_level.json \
  --pcf constraints/LatticeiCE40HX8K.pcf \
  --pcf-allow-unconstrained \
  --asc top_level.asc

icetime -d hx8k -mtr top_level.rpt top_level.asc
icepack -n top_level.asc top_level_noinit.bin
iceprog -S top_level_noinit.bin
```

For convenience, the long build and capture commands were wrapped in local Makefile targets:

```
make alp-clean
make alp-build
make alp-program
make alp-capture OUT=puf_data.txt
make alp-check OUT=puf_data.txt
```

The combined one-step flow is:

```
make alp-run OUT=puf_data.txt
make alp-check OUT=puf_data.txt
```

The diagnostic UART mode can be run with:

```
make alp-run REQUEST=d OUT=debug_packet_16k.txt
```

11 Measurement and Analysis Notes

For final measurements, the temporary writeback mode must remain disabled. The board should be power-cycled between captures, because the SRAM/BRAM power-up state is the physical source of the PUF response.

The collected files follow the model used in the task description:

- one capture file corresponds to one power cycle;
- each file contains 512 lines;
- each line contains 64 hexadecimal characters, i.e. 256 bits;
- the same line number across multiple captures corresponds to the same simulated PUF instance across different power cycles.

The analysis computes Uniformity, Bit-aliasing, Uniqueness, and Reliability on individual bits, not only on bytes. The measured average metrics were close to the expected SRAM PUF behavior: Uniformity, Bit-aliasing, and Uniqueness were close to the ideal 50%, while Reliability was below the ideal 100% due to unstable bits across power cycles.

12 Short Defense Summary

A concise explanation to a teaching assistant could be:

The module waits for a UART request byte. On `s/S`, it starts reading the SRAM/BRAM address space through `combined_ram`. The RAM provides 16-bit words, but UART transmits 8-bit bytes, so I use `i_r` as a byte counter: `i_r[13:1]` becomes the RAM word address and `i_r[0]` selects the low or high byte. The selected byte is first stored in `puf_byte_reg`, then sent through UART using a ready/enable handshake. `UART_SEND` generates a registered one-clock `uart_tx_enable` pulse. Then `UART_WAIT_FINISH` first waits until the UART ready signal goes low, confirming that the byte was accepted, and only then waits for it to return high before the FSM continues. This loop repeats until 16384 bytes are transmitted, corresponding to 512 PUF instances of 256 bits each. The debug request mode and writeback mode were temporary tools used to separate UART/FSM issues from RAM readout issues.